

# Opis użycia bibliotek

## Standardowe biblioteki

- abc - definiowanie klas i metod abstrakcyjnych
- collections - deque (do przechowywania kolejki Tabu w algorytmie Tabu) i defaultdict
- concurrent.futures - ThreadPoolExecutor potrzebny do wywołania kodu w Tabu równolegle
- dataclasses - do definiowania klas danych, których strukturę można zamrozić i łatwo porównywać
- datetime i time - do pracy z czasami
- enum - do wyboru kryterium czasu lub przesiadek
- functools - bardzo ważne cache'owanie za pomocą lru\_cache, znacznie powtarzające się obliczenia
- heapq - przydatna struktura danych, pozwalająca na szybkie sortowanie danych
- itertools - do generowania liczników
- os - do pracy z systemem plików
- pickle - do serializacji i deserializacji obiektów
- random - do generowania losowych liczb
- re - do pracy z wyrażeniami regularnymi
- sys - do zarządzania strumieniami wejścia/wyjścia

## Zewnętrzne biblioteki

- geopy - do obliczania odległości między przystankami
- pandas - do szybszego przetworzenia pliku CSV

```
In [81]: from datetime import time
import re

def time_to_seconds(t: time) -> int:
    return t.hour * 3600 + t.minute * 60 + t.second

def seconds_to_time(s: int) -> time:
    h, remainder = divmod(s, 3600)
    m, s = divmod(remainder, 60)
    return time(h % 24, m, s)

def convert_to_24_hour_time(time_to_normalize: str) -> time:
    match: re.Match[str] | None = re.match(
        r"(\d{2}):(\d{2}):(\d{2})", time_to_normalize)
    if not match:
        raise ValueError(f"Invalid time format: {time_to_normalize}")

    hour, minute, second = map(int, match.groups())

    if hour >= 24:
```

```

        hour -= 24

    return time(hour, minute, second)

```

Najpierw trzeba zacząć od różnych funkcji pomocniczych. Jedną z nich będzie oczyszczenie danych - kursy kończące się po północy mają godziny wychodzące poza standard 24 godzin (o maksymalnie parę godzin), więc aby działały poprawnie ze standardowymi bibliotekami czasu Python, musiały być znormalizowane.

```

In [82]: from dataclasses import dataclass
        from geopy.point import Point
        from collections import defaultdict
        import sys
        import enum

        @dataclass
        class Node:
            name: str
            location: Point
            _hash: int = 0

            def __post_init__(self) -> None:
                self._hash = hash((self.name, self.location.format_unicode()))

            def __hash__(self) -> int:
                return self._hash

        @dataclass
        class CommunicationStep:
            company: str
            line: str
            departure_time: time
            arrival_time: time
            start_stop: Node
            end_stop: Node

            @staticmethod
            def from_parsed_csv_line(row: list[str]) -> "CommunicationStep":
                start_stop_point: Point = Point(latitude=row[7], longitude=row[8])
                start_stop = Node(row[5], start_stop_point)

                end_stop_point: Point = Point(latitude=row[9], longitude=row[10])
                end_stop = Node(row[6], end_stop_point)

                company, line, departure_str, arrival_str = row[1:5]

                departure_time: time = convert_to_24_hour_time(departure_str)
                arrival_time: time = convert_to_24_hour_time(arrival_str)

                new_communication_step = CommunicationStep(
                    company, line, departure_time, arrival_time, start_stop, end_stop)

                return new_communication_step

            def __str__(self):
                return f"line {self.line} | {self.start_stop} {self.departure_time} -> {"

```

```

def __eq__(self, other: object) -> bool:
    if not isinstance(other, CommunicationStep):
        return False
    return (
        self.company == other.company and
        self.line == other.line and
        self.departure_time == other.departure_time and
        self.arrival_time == other.arrival_time and
        self.start_stop == other.start_stop and
        self.end_stop == other.end_stop
    )

def __hash__(self) -> int:
    return hash((self.company, self.line, self.departure_time, self.arrival_

class Graph:
    def __init__(self) -> None:
        self.nodes: dict[str, Node] = {}
        self.edges: dict[tuple[str, str],
                           set[CommunicationStep]] = defaultdict(set)
        self.adjacency_list: dict[str,
                                   set[CommunicationStep]] = defaultdict(set)

@dataclass
class LineStep:
    start_node_name: str
    end_node_name: str
    line: str
    start_time: time
    end_time: time

@dataclass
class Path:
    steps: list[LineStep]
    cost: float
    calculation_time: float

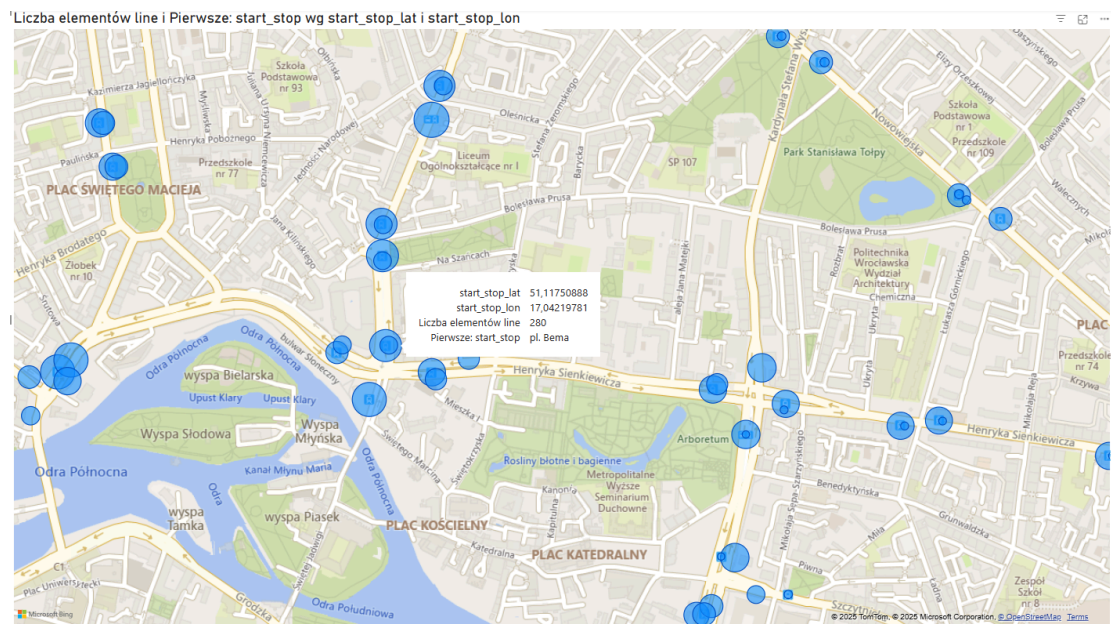
    def pretty_print(self) -> None:
        print("Schedule:")
        for step in self.steps:
            print(
                f"{step.line}\t| {step.start_node_name} {step.start_time} -> {st
        print(f"Total cost: {self.cost} units", file=sys.stderr, flush=True)
        print(
            f"Execution time: {self.calculation_time:.4f} seconds", file=sys.std

class NoPathFoundError(Exception):
    """Raised when no path is found between the start and end nodes."""
    pass

class OptimizationCriterion(enum.Enum):
    TIME = "time"
    TRANSFERS = "transfers"

```

Modele konieczne do ekstrakcji danych z pliku .csv i ustrukturyzowanie ich w postaci grafu. Jedną z najważniejszych klas jest CommunicationStep - krawędź w grafie przechowująca wiele informacji o linii, czasie odjazdu, przyjazdu i przystankach.



Jak widać na wizualizacji, wiele przystanków ma tę samą nazwę, ale różną lokalizację. Ostatecznie wszystkie połączenia między przystankami w grafie będą bazowane tylko na nazwach przystanków (klucze w słowniku), a nie konkretnych instancjach Node. Ponieważ lokalizacje zawsze są blisko siebie dla tej samej nazwy przystanku, nie wpływa to na działanie algorytmów.

## Struktura grafów

Graf ma dość prostą strukturę. Jest słownik instancji Node od nazwy przystanku, słownik listy połączeń komunikacyjnych (CommunicationStep) od nazw przystanków i słownik listy połączeń komunikacyjnych od nazwy przystanku, z którego pochodzą.

## Inne

LineStep to cała określona trasa jednej linii z A do B. Path to zbiór tych konkretnych tras.

Dodatkowo zastosowano klasę enum dla kryterium optymalizacyjnego.

W przypadku braku znalezienia trasy algorytmy wywołują wyjątek NoPathFoundError.

```
In [83]: from geopy.distance import geodesic
from functools import lru_cache

@lru_cache(maxsize=None)
def distance_heuristic(node1: Node, node2: Node) -> float:
    return geodesic(node1.location, node2.location).kilometers ** 2

def transfer_heuristic(transfer_count: int) -> float:
    transfer_penalty_weight = 500000
    return transfer_count * transfer_penalty_weight
```

```

def clear_caches() -> None:
    distance_heuristic.cache_clear()

def post_clear_cache(func):
    def wrapper(*args, **kwargs) -> None:
        result = func(*args, **kwargs)
        clear_caches()
        return result
    return wrapper

def generate_path(start: str, path_taken: list[CommunicationStep], elapsed: float,
    path_steps: list[LineStep] = []):

    none_time: time = time(0, 0)

    step: CommunicationStep = path_taken[0]
    last_line: str = step.line
    path_steps.append(LineStep(
        start, "", step.line, step.departure_time, none_time))

    for i, step in enumerate(path_taken[:-1]):
        if step.line == last_line:
            continue
        last_line = step.line

        path_steps[-1].end_time = path_taken[i - 1].arrival_time
        path_steps[-1].end_node_name = step.start_stop.name

        path_steps.append(LineStep(step.start_stop.name,
            "", last_line, step.departure_time, none_time))

    step = path_taken[-1]
    path_steps[-1].end_time = step.arrival_time
    path_steps[-1].end_node_name = step.end_stop.name

    path: Path = Path(path_steps, cost, elapsed)
    return path

```

Funkcje heurystyczne dla algorytmu A\* oraz pomocnicze do konwersji obliczonej trasy na Path. Aby nie zapewnić przewagi algorytmom wywoływanym później, czyści się cache.

```

In [84]: from dataclasses import dataclass, field
import heapq
from itertools import count
import time as t

@dataclass(order=True)
class QueueEntry:
    priority: int
    counter: int
    current_stop_name: str = field(compare=False)
    path_taken: list[CommunicationStep] = field(compare=False)
    current_time_sec: int = field(compare=False)

```

```

@post_clear_cache
def dijkstra(start: str, end: str, start_time: time, graph: Graph) -> Path:
    if start not in graph.nodes:
        raise ValueError("Start stop does not exist in the graph.")
    if end not in graph.nodes:
        raise ValueError("End stop does not exist in the graph.")

    start_node: Node = graph.nodes[start]
    end_node: Node = graph.nodes[end]

    start_time_sec: int = time_to_seconds(start_time)

    queue: list[QueueEntry] = []
    counter = count()
    heapq.heappush(queue, QueueEntry(0, next(counter),
                                      start_node.name, [], start_time_sec))

    visited: set[str] = set()

    start_time_perf: float = t.perf_counter()

    while queue:
        entry: QueueEntry = heapq.heappop(queue)

        if entry.current_stop_name in visited:
            continue
        visited.add(entry.current_stop_name)

        if entry.current_stop_name == end_node.name:
            elapsed: float = t.perf_counter() - start_time_perf

            path: Path = generate_path(
                start, entry.path_taken, elapsed, entry.priority)

            return path

        for (start_name, end_name), steps in graph.edges.items():
            if start_name != entry.current_stop_name:
                continue
            for step in steps:
                departure_seconds: int = time_to_seconds(step.departure_time)
                arrival_seconds: int = time_to_seconds(step.arrival_time)

                if departure_seconds >= entry.current_time_sec:
                    travel_time: int = arrival_seconds - entry.current_time_sec
                    if travel_time < 0:
                        travel_time += 86400 # handle crossing midnight

                    new_priority: int = entry.priority + travel_time
                    new_queue_entry = QueueEntry(
                        new_priority,
                        next(counter),
                        end_name,
                        entry.path_taken + [step],
                        arrival_seconds
                    )
                    heapq.heappush(queue, new_queue_entry)

    raise NoPathFoundError(f"No path found from {start} to {end}")

```

Algorytm Dijkstra - wylicza najszybszą pod względem czasu trasę między A i B. Bez dodatkowej wiedzy o potencjale rozwiązań, oblicza dystans dla większej liczby wierzchołków niż A\*. Algorytm działa, przeszukując ciągle krawędzie sąsiednie pierwszej krawędzi, a potem ich sąsiadów i tak dalej. Dzięki wykorzystaniu struktury danych heap, rozwiązania są szybko sortowane pod względem czasu odjazdu.

```
In [85]: import pandas as pd
import pickle
import os

csv_filename = "connection_graph"
separator = ","
skipfooter = 0
graph: Graph

def try_load_from_pickle() -> Graph | None:
    graph: Graph | None = None

    if os.path.exists(f"data/{csv_filename}.pkl"):
        with open(f"data/{csv_filename}.pkl", "rb") as f:
            graph = pickle.load(f)
        print("Graph loaded from graph.pkl.")
    if os.path.exists(f"lab01/data/{csv_filename}.pkl"):
        with open(f"lab01/data/{csv_filename}.pkl", "rb") as f:
            graph = pickle.load(f)
        print("Graph loaded from graph.pkl.")

    return graph

def load_from_csv() -> Graph:
    graph: Graph

    df: pd.DataFrame
    if os.path.exists(f"data/{csv_filename}.csv"):
        df = pd.read_csv(f"data/{csv_filename}.csv", encoding="utf-8",
                        sep=separator, skipfooter=skipfooter, engine="python")
    else:
        df = pd.read_csv(f"lab01/data/{csv_filename}.csv", encoding="utf-8",
                        sep=separator, skipfooter=skipfooter, engine="python")

    graph = Graph()

    for _, row in df.iterrows():
        step: CommunicationStep = CommunicationStep.from_parsed_csv_line(
            list(row))

        for stop in [step.start_stop, step.end_stop]:
            if stop.name not in graph.nodes:
                graph.nodes[stop.name] = stop
                graph.adjacency_list[stop.name] = set()

        key: tuple[str, str] = (step.start_stop.name, step.end_stop.name)
        if step not in graph.edges[key]:
            graph.edges[key].add(step)
            graph.adjacency_list[step.start_stop.name].add(step)
```

```

with open(f"data/{csv_filename}.pkl", "wb") as f2:
    pickle.dump(graph, f2)

print("Done parsing .csv")

return graph

def load_from_pickle_fallback_csv() -> Graph:
    graph: Graph | None = try_load_from_pickle()

    if graph is not None:
        return graph

    graph = load_from_csv()

    return graph

```

Ponieważ raz obliczony graf można wykorzystać potem wiele razy - algorytmy nie zmieniają jego stanu - postanowiono zapisać go do pliku za pomocą Pickle. Znacznie przyspieszyło to czas testowania algorytmów (kod przeniesiono do notebooka dopiero potem).

```
In [86]: graph: Graph = load_from_pickle_fallback_csv()
```

Graph loaded from graph.pkl.

```
In [87]: from datetime import datetime

a = "Prusa"
b = "PORT LOTNICZY"
optimization_criterion_str = "p"
start_time = "08:00:00"

start_time_obj: time = datetime.strptime(start_time, "%H:%M:%S").time()

path: Path = dijkstra(a, b, start_time_obj, graph)

path.pretty_print()
```

Total cost: 3420 units  
Execution time: 3.1011 seconds

Schedule:

```

1      | Prusa 08:02:00 -> Wyszyńskiego 08:03:00
128    | Wyszyńskiego 08:03:00 -> Dubois 08:09:00
6      | Dubois 08:10:00 -> Rynek 08:15:00
12     | Rynek 08:15:00 -> PL. JANA PAWŁA II 08:17:00
122    | PL. JANA PAWŁA II 08:17:00 -> pl. Orłąt Lwowskich 08:19:00
148    | pl. Orłąt Lwowskich 08:20:00 -> Nowodworska 08:30:00
134    | Nowodworska 08:30:00 -> Rogowska (P+R) 08:34:00
132    | Rogowska (P+R) 08:35:00 -> MIŃSKA (Rondo Rotm. Pileckiego) 08:37:00
106    | MIŃSKA (Rondo Rotm. Pileckiego) 08:43:00 -> PORT LOTNICZY 08:57:00

```

```
In [88]: from dataclasses import dataclass, field
import heapq
from itertools import count
import time as t
```



```

@dataclass(order=True)
class QueueEntry:
    priority: float
    counter: int
    current_stop_name: str = field(compare=False)
    path_taken: list[CommunicationStep] = field(compare=False)
    current_time_sec: int = field(compare=False)
    transfer_count: int = field(compare=False)

def should_skip(entry: QueueEntry, best_state: dict[str, tuple[int, int]]) -> bool:
    current_state: tuple[int, int] | None = best_state.get(
        entry.current_stop_name)
    if current_state is None:
        return False
    best_transfers, best_arrival = current_state
    return (entry.transfer_count > best_transfers) or \
        (entry.transfer_count ==
         best_transfers and entry.current_time_sec >= best_arrival)

def update_best_state(entry: QueueEntry, best_state: dict[str, tuple[int, int]])
    best_state[entry.current_stop_name] = (
        entry.transfer_count, entry.current_time_sec)

def calculate_priority(entry: QueueEntry, current_node: Node, neighbor_node: Node,
    travel_time: int, optimization_criterion: OptimizationCriterion) -> tuple[float, int]:
    cost_so_far: float = entry.priority + travel_time
    estimated_remaining: float = 0
    new_transfer_count: int = entry.transfer_count

    if optimization_criterion == OptimizationCriterion.TIME:
        cost_so_far -= distance_heuristic(current_node, neighbor_node)
        estimated_remaining = distance_heuristic(neighbor_node, neighbor_node)

    if optimization_criterion == OptimizationCriterion.TRANSFERS:
        cost_so_far += transfer_heuristic(entry.transfer_count)
        if entry.path_taken:
            previous_step: CommunicationStep = entry.path_taken[-1]
            is_transfer: bool = previous_step.line != step.line
            new_transfer_count += 1 if is_transfer else 0

    total_priority: float = cost_so_far + estimated_remaining
    return total_priority, new_transfer_count

@post_clear_cache
def a_star_search(start: str, end: str, start_time: time, graph: Graph, optimization_criterion: OptimizationCriterion) -> tuple[Node, list[CommunicationStep], int]:
    if start not in graph.nodes or end not in graph.nodes:
        raise ValueError("Start or end stop does not exist in the graph.")

    start_node: Node = graph.nodes[start]
    end_node: Node = graph.nodes[end]
    start_time_sec: int = time_to_seconds(start_time)

    queue: list[QueueEntry] = []
    counter = count()
    initial_heuristic: float = distance_heuristic(

```

```

        start_node, end_node) if optimization_criterion == OptimizationCriterion

    heapq.heappush(queue, QueueEntry(initial_heuristic, next(
        counter), start_node.name, [], start_time_sec, 0))
    best_state: dict[str, tuple[int, int]] = {}
    start_time_perf: float = t.perf_counter()

    while queue:
        entry: QueueEntry = heapq.heappop(queue)
        if should_skip(entry, best_state):
            continue
        update_best_state(entry, best_state)
        if entry.current_stop_name == end_node.name:
            elapsed: float = t.perf_counter() - start_time_perf
            return generate_path(start, entry.path_taken, elapsed, entry.priorit

    current_node: Node = graph.nodes[entry.current_stop_name]
    for (start_name, end_name), steps in graph.edges.items():
        if start_name != entry.current_stop_name:
            continue
        for step in steps:
            departure_seconds: int = time_to_seconds(step.departure_time)
            arrival_seconds: int = time_to_seconds(step.arrival_time)

            if departure_seconds >= entry.current_time_sec:
                # handle midnight wrap
                travel_time: int = (
                    arrival_seconds - entry.current_time_sec) % 86400

            neighbor_node: Node = graph.nodes[end_name]
            total_priority, new_transfer_count = calculate_priority(
                entry, current_node, neighbor_node, step, travel_time, c
            )
            new_entry = QueueEntry(
                total_priority,
                next(counter),
                end_name,
                entry.path_taken + [step],
                arrival_seconds,
                new_transfer_count
            )
            heapq.heappush(queue, new_entry)

    raise NoPathFoundError(f"No path found from {start} to {end}")

```

Dla A\* można zoptymalizować trasę pod liczbę przesiadek lub czas dojazdu. Jest to możliwe dzięki heurystyce - dla optymalizacji czasu można użyć heurystyki obliczającej odległość między aktualnym przystankiem a celem. Dzięki temu jest gwarancja, że priorytet będą miały przystanki fizycznie bliżej na mapie do przystanku końcowego. Nie zawsze są to najlepsze przystanki, ale zazwyczaj są. Dla optymalizacji przesiadek wykorzystano heurystykę nakładającą duży koszt na każdą przesiadkę. Przez narzut metody geodesic dla niektórych tras Dijkstra może szybciej wyliczyć trasę (mimo iż musi sprawdzić znacznie większą liczbę połączeń).

```

In [89]: a = "Nowy Dom"
         b = "PORT LOTNICZY"
         optimization_criterion_str = "p"

```

```

start_time = "08:00:00"

start_time_obj: time = datetime.strptime(start_time, "%H:%M:%S").time()
optimization_criterion: OptimizationCriterion = OptimizationCriterion.TIME

path: Path = a_star_search(a, b, start_time_obj, graph, optimization_criterion)
path.pretty_print()

optimization_criterion: OptimizationCriterion = OptimizationCriterion.TRANSFERS
path: Path = a_star_search(a, b, start_time_obj, graph, optimization_criterion)
path.pretty_print()

```

Total cost: 4025.301216172097 units

Execution time: 1.0558 seconds

Schedule:

```

120      | Nowy Dom 08:14:00 -> Na Niskich łąkach 08:21:00
114      | Na Niskich łąkach 08:21:00 -> Świstackiego 08:22:00
120      | Świstackiego 08:22:00 -> Komuny Paryskiej 08:23:00
114      | Komuny Paryskiej 08:23:00 -> Komuny Paryskiej (szkoła) 08:24:00
120      | Komuny Paryskiej (szkoła) 08:24:00 -> Wzgórze Partyzantów 08:25:00
106      | Wzgórze Partyzantów 08:26:00 -> Wrocławski Park Przemysłowy 08:37:00
13       | Wrocławski Park Przemysłowy 08:37:00 -> Park Biznesu 08:39:00
106      | Park Biznesu 08:39:00 -> Babimojska 08:40:00
13       | Babimojska 08:40:00 -> Strzegomska 148 08:41:00
106      | Strzegomska 148 08:41:00 -> Strzegomska (krzyżówka) 08:44:00
124      | Strzegomska (krzyżówka) 08:44:00 -> Rogowska (P+R) 08:45:00
122      | Rogowska (P+R) 08:46:00 -> MIŃSKA (Rondo Rotm. Pileckiego) 08:48:00
106      | MIŃSKA (Rondo Rotm. Pileckiego) 08:48:00 -> PORT LOTNICZY 09:03:00

```

Total cost: 11003780 units

Execution time: 7.2140 seconds

Schedule:

```

120      | Nowy Dom 08:14:00 -> Wzgórze Partyzantów 08:25:00
106      | Wzgórze Partyzantów 08:26:00 -> PORT LOTNICZY 09:03:00

```

```

In [93]: from random import randint
         from abc import ABC, abstractmethod
         from models import Graph, Node
         from geopy.distance import geodesic

         class TabuSizeStrategy(ABC):
             @abstractmethod
             def get_tabu_size(self, required_stops: list[str]) -> int: ...

         class FixedTabuSizeStrategy(TabuSizeStrategy):
             def __init__(self, size: int = 10) -> None:
                 self.size: int = size

             def get_tabu_size(self, required_stops: list[str]) -> int:
                 return self.size

         class DynamicTabuSizeStrategy(TabuSizeStrategy):
             def __init__(self, k: float = 5.0, min_size: int = 10) -> None:
                 self.k: float = k
                 self.min_size: int = min_size

             def get_tabu_size(self, required_stops: list[str]) -> int:
                 return max(self.min_size, int(self.k * len(required_stops)))

```

```

class NeighborhoodSamplingStrategy(ABC):
    @abstractmethod
    def generate_swaps(self, num_stops: int) -> list[tuple[int, int]]: ...

class FullSamplingStrategy(NeighborhoodSamplingStrategy):
    def generate_swaps(self, num_stops: int) -> list[tuple[int, int]]:
        swaps: list[tuple[int, int]] = []
        for i in range(1, num_stops):
            for j in range(i + 1, num_stops + 1):
                swaps.append((i, j))
        return swaps

class RandomSamplingStrategy(NeighborhoodSamplingStrategy):
    def __init__(self, sample_size: int = 10) -> None:
        self.sample_size: int = sample_size

    def generate_swaps(self, num_stops: int) -> list[tuple[int, int]]:
        swaps: set[tuple[int, int]] = set()
        while len(swaps) < min(self.sample_size, (num_stops * (num_stops - 1)) / 2):
            i: int = randint(1, num_stops - 1)
            j: int = randint(i + 1, num_stops)
            swaps.add((i, j))
        return list(swaps)

class AspirationStrategy(ABC):
    @abstractmethod
    def allow_route(
        self, tabu_set: set[tuple[str]], route: tuple[str, ...]
    ) -> bool: ...

class StrictTabuAspirationStrategy(AspirationStrategy):
    def allow_route(self, tabu_set, route) -> bool:
        return route not in tabu_set

class AllowTabuAspirationStrategy(AspirationStrategy):
    def allow_route(self, tabu_set, route) -> bool:
        return True

class FirstPathStrategy(ABC):
    @abstractmethod
    def calculate_first_path(
        self, start: str, route: list[str], graph: Graph
    ) -> list[str]: ...

class OrderedFirstPathStrategy(FirstPathStrategy):
    def calculate_first_path(
        self, start: str, route: list[str], graph: Graph
    ) -> list[str]:
        return [start] + route + [start]

```

```

class EstimateClosestFirstPathStrategy(FirstPathStrategy):
    def calculate_first_path(
        self, start: str, route: list[str], graph: Graph
    ) -> list[str]:
        nodes: list[Node] = [graph.nodes[stop] for stop in route]

        start_node: Node = graph.nodes[start]

        current_stop: Node = start_node

        path: list[str] = [start]

        while len(nodes) > 0:
            nodes.sort(
                key=lambda node: geodesic(
                    current_stop.location, node.location).km
            )

            path.append(nodes[0].name)
            current_stop = nodes.pop(0)

        path.append(start)

        return path

```

## Strategia wielkości Tabu

Można wybrać strategię z określoną z góry wielkością listy Tabu lub taką, która liniowo określa wielkość listy Tabu na podstawie liczby przystanków do odwiedzenia.

## Strategia dobierania sąsiedztwa

Można wybrać strategię wszystkich możliwych swapów między kolejnością odwiedzania przystanków w Tabu lub tylko losowo wybranej części.

## Strategia aspiracji

Można zezwolić na aspirację - wybór rozwiązania znajdującego się w liście Tabu, jeśli jest najlepsze - lub ściśle przestrzeganie listy Tabu.

## Strategia wyboru pierwszej trasy

Można wybrać trasę tak jak użytkownik podał, albo próba obliczenia potencjalnie optymalniejszej na podstawie odległości między przystankami.

In [94]:

```

from collections import deque
from concurrent.futures import ThreadPoolExecutor
from datetime import timedelta

```

```

@lru_cache(maxsize=None)
def get_shortest_path(
    start: str,
    end: str,
    start_time_sec: int,

```

```

graph: Graph,
optimization_criterion: OptimizationCriterion,
) -> Path:
    start_time_obj: time = (
        datetime.min + timedelta(seconds=start_time_sec)).time()
    path: Path = a_star_search(
        start, end, start_time_obj, graph, optimization_criterion
    )
    return path

@dataclass(order=True)
class TabuSolution:
    cost: int
    route: list[str] = field(compare=False)
    steps: list[CommunicationStep] = field(compare=False)

def post_clear_cache(func):
    def wrapper(*args, **kwargs) -> None:
        result = func(*args, **kwargs)
        get_shortest_path.cache_clear()
        return result

    return wrapper

def calculate_total_cost_and_steps(
    route: list[str],
    start_time: time,
    graph: Graph,
    optimization_criterion: OptimizationCriterion,
):
    total_cost: float = 0
    steps: list[LineStep] = []
    current_time: time = start_time

    for i in range(len(route) - 1):
        start: str = route[i]
        end: str = route[i + 1]

        start_time_sec: int = time_to_seconds(current_time)
        try:
            path: Path = get_shortest_path(
                start, end, start_time_sec, graph, optimization_criterion
            )
        except NoPathFoundError:
            return float("inf"), None
        total_cost += path.cost
        steps += path.steps

        current_time = path.steps[-1].end_time if path.steps else current_time

    return total_cost, steps

def estimate_good_first_path(start: str, route: list[str], graph: Graph) -> list
nodes: list[Node] = [graph.nodes[stop] for stop in route]

start_node: Node = graph.nodes[start]

```

```

current_stop: Node = start_node

path: list[str] = [start]

while len(nodes) > 0:
    nodes.sort(key=lambda node: geodesic(
        current_stop.location, node.location).km)

    path.append(nodes[0].name)
    current_stop = nodes.pop(0)

path.append(start)

return path

@post_clear_cache
def tabu_search(
    start: str,
    required_stops: list[str],
    start_time: time,
    graph: Graph,
    optimization_criterion: OptimizationCriterion,
    max_iterations=5,
    tabu_size_strategy: TabuSizeStrategy = FixedTabuSizeStrategy(),
    sampling_strategy: NeighborhoodSamplingStrategy = FullSamplingStrategy(),
    aspiration_strategy: AspirationStrategy = StrictTabuAspirationStrategy(),
    first_path_strategy: FirstPathStrategy = EstimateClosestFirstPathStrategy()
) -> Path:
    tabu_size: int = tabu_size_strategy.get_tabu_size(required_stops)

    tabu_set: set[tuple] = set()
    tabu_queue: deque[tuple] = deque()

    start_time_perf: float = t.perf_counter()

    current_route: list[str] = first_path_strategy.calculate_first_path(
        start, required_stops, graph)
    best_route: list[str] = current_route.copy()

    best_cost, best_steps = calculate_total_cost_and_steps(
        best_route, start_time, graph, optimization_criterion
    )

    for _ in range(max_iterations):
        neighborhood: list[TabuSolution] = []
        swaps: list[tuple[int, int]] = sampling_strategy.generate_swaps(
            len(required_stops)
        )

        with ThreadPoolExecutor(max_workers=8) as executor:
            futures = []
            for i, j in swaps:
                new_route: list[str] = current_route.copy()
                new_route[i], new_route[j] = new_route[j], new_route[i]
                route_tuple: tuple[str, ...] = tuple(new_route)

                if not aspiration_strategy.allow_route(tabu_set, route_tuple):
                    continue

```

```

        futures.append(
            executor.submit(
                calculate_total_cost_and_steps,
                new_route,
                start_time,
                graph,
                optimization_criterion,
            )
        )

    for future, (i, j) in zip(futures, swaps):
        cost, steps = future.result()
        new_route = current_route.copy()
        new_route[i], new_route[j] = new_route[j], new_route[i]
        route_tuple = tuple(new_route)

        if (route_tuple not in tabu_set) or (cost < best_cost * 0.99):
            heapq.heappush(neighborhood, TabuSolution(
                cost, new_route, steps))

    if not neighborhood:
        break

    best_neighbor: TabuSolution = heapq.heappop(neighborhood)

    if best_neighbor.cost < best_cost:
        best_cost = best_neighbor.cost
        best_route = best_neighbor.route
        best_steps = best_neighbor.steps

    current_route = best_neighbor.route
    route_tuple = tuple(current_route)
    tabu_set.add(route_tuple)
    tabu_queue.append(route_tuple)

    if len(tabu_queue) > tabu_size:
        oldest = tabu_queue.popleft()
        tabu_set.remove(oldest)

    elapsed: float = t.perf_counter() - start_time_perf

    return Path(best_steps, best_cost, elapsed)

```

Algorytm Tabu - za pomocą algorytmu A\* generuje ścieżki między kolejnymi przystankami do odwiedzenia. Następnie szuka lokalnie lepszego rozwiązania, aż osiągnie maksymalną liczbę iteracji. Dodatkowo wykorzystano estymowaną pierwszą dobrą ścieżkę, szukając kolejno najbliższych przystanków do poprzedniego. Dodatkowo wykorzystano ThreadPoolExecutor do zrównoleglenia obliczeń kosztów ścieżek dla swapów.

```

In [95]: stops: list[str] = ["C.H. Korona", "FAT", "GAJ"]

print("Tabu czas")
path: Path = tabu_search(a, stops, start_time_obj, graph, OptimizationCriterion.
                        sampling_strategy=RandomSamplingStrategy(3), first_path
path.pretty_print()

```



```

print("Tabu przesiadki")
path = tabu_search(a, stops, start_time_obj, graph,
                  OptimizationCriterion.TRANSFERS, sampling_strategy=RandomSamplingStrategy)
path.pretty_print()

print("Tabu czas z aspiracją")
path = tabu_search(a, stops, start_time_obj, graph,
                  OptimizationCriterion.TIME, sampling_strategy=RandomSamplingStrategy)
path.pretty_print()

print("Tabu dynamiczna wielkość listy Tabu")
path = tabu_search(a, stops, start_time_obj, graph, OptimizationCriterion.TIME,
                  tabu_size_strategy=DynamicTabuSizeStrategy(k=1.0, min_size=1))
path.pretty_print()

print("Tabu nieskończona wielkość listy Tabu")
path = tabu_search(a, stops, start_time_obj, graph,
                  OptimizationCriterion.TIME, tabu_size_strategy=FixedTabuSizeStrategy)
path.pretty_print()

```

#### Tabu czas

Total cost: 8306.878786135247 units

Execution time: 22.0532 seconds

#### Schedule:

```

120      | Nowy Dom 08:14:00 -> Na Niskich łąkach 08:21:00
3        | Na Niskich łąkach 08:22:00 -> Armii Krajowej 08:26:00
134      | Armii Krajowej 08:28:00 -> Armii Krajowej (Bogedaina) 08:29:00
143      | Armii Krajowej (Bogedaina) 08:30:00 -> GAJ 08:36:00
143      | GAJ 08:36:00 -> Działkowa 08:38:00
K        | Działkowa 08:39:00 -> Śliczna 08:41:00
143      | Śliczna 08:41:00 -> Uniwersytet Ekonomiczny 08:44:00
134      | Uniwersytet Ekonomiczny 08:44:00 -> Bzowa (Centrum Historii Zajeżdźnia)
08:55:00
11       | Bzowa (Centrum Historii Zajeżdźnia) 08:55:00 -> FAT 08:58:00
134      | FAT 08:58:00 -> Stalowa 09:03:00
4        | Stalowa 09:04:00 -> DWORZEC GŁÓWNY 09:14:00
N        | DWORZEC GŁÓWNY 09:14:00 -> GALERIA DOMINIKAŃSKA 09:18:00
13       | GALERIA DOMINIKAŃSKA 09:19:00 -> PL. GRUNWALDZKI 09:26:00
116      | PL. GRUNWALDZKI 09:28:00 -> Kochanowskiego 09:30:00
131      | Kochanowskiego 09:30:00 -> Śniadeckich 09:31:00
116      | Śniadeckich 09:31:00 -> Zacisze 09:32:00
131      | Zacisze 09:32:00 -> C.H. Korona 09:38:00
111      | C.H. Korona 09:38:00 -> Zacisze 09:43:00
131      | Zacisze 09:43:00 -> Śniadeckich 09:44:00
111      | Śniadeckich 09:44:00 -> Bujwida 09:48:00
131      | Bujwida 09:48:00 -> PL. GRUNWALDZKI 09:50:00
115      | PL. GRUNWALDZKI 09:50:00 -> Kliniki - Politechnika Wrocławska 09:52:00
10       | Kliniki - Politechnika Wrocławska 09:53:00 -> Chełmońskiego 09:58:00
143      | Chełmońskiego 10:05:00 -> Międzyrzecka 10:08:00
920      | Międzyrzecka 10:12:00 -> Nowy Dom 10:16:00

```

#### Tabu przesiadki

Total cost: 3577460 units

Execution time: 42.8382 seconds

Schedule:

120 | Nowy Dom 08:14:00 -> Międzyrzecka 08:18:00  
143 | Międzyrzecka 08:27:00 -> GAJ 08:36:00  
143 | GAJ 08:36:00 -> C.H. Korona 09:37:00  
143 | C.H. Korona 09:37:00 -> FAT 13:53:00  
143 | FAT 13:53:00 -> Międzyrzecka 00:18:00  
120 | Międzyrzecka 05:27:00 -> Nowy Dom 05:31:00

Tabu czas z aspiracją

Total cost: 8306.878786135247 units

Execution time: 19.4758 seconds

Schedule:

120 | Nowy Dom 08:14:00 -> Na Niskich łąkach 08:21:00  
3 | Na Niskich łąkach 08:22:00 -> Armii Krajowej 08:26:00  
134 | Armii Krajowej 08:28:00 -> Armii Krajowej (Bogedaina) 08:29:00  
143 | Armii Krajowej (Bogedaina) 08:30:00 -> GAJ 08:36:00  
143 | GAJ 08:36:00 -> Działkowa 08:38:00  
K | Działkowa 08:39:00 -> Śliczna 08:41:00  
143 | Śliczna 08:41:00 -> Uniwersytet Ekonomiczny 08:44:00  
134 | Uniwersytet Ekonomiczny 08:44:00 -> Bzowa (Centrum Historii Zajeżdźnia)  
08:55:00  
11 | Bzowa (Centrum Historii Zajeżdźnia) 08:55:00 -> FAT 08:58:00  
134 | FAT 08:58:00 -> Stalowa 09:03:00  
4 | Stalowa 09:04:00 -> DWORZEC GŁÓWNY 09:14:00  
N | DWORZEC GŁÓWNY 09:14:00 -> GALERIA DOMINIKAŃSKA 09:18:00  
13 | GALERIA DOMINIKAŃSKA 09:19:00 -> PL. GRUNWALDZKI 09:26:00  
116 | PL. GRUNWALDZKI 09:28:00 -> Kochanowskiego 09:30:00  
131 | Kochanowskiego 09:30:00 -> Śniadeckich 09:31:00  
116 | Śniadeckich 09:31:00 -> Zacisze 09:32:00  
131 | Zacisze 09:32:00 -> C.H. Korona 09:38:00  
111 | C.H. Korona 09:38:00 -> Zacisze 09:43:00  
131 | Zacisze 09:43:00 -> Śniadeckich 09:44:00  
111 | Śniadeckich 09:44:00 -> Bujwida 09:48:00  
131 | Bujwida 09:48:00 -> PL. GRUNWALDZKI 09:50:00  
115 | PL. GRUNWALDZKI 09:50:00 -> Kliniki - Politechnika Wrocławska 09:52:00  
10 | Kliniki - Politechnika Wrocławska 09:53:00 -> Chełmońskiego 09:58:00  
143 | Chełmońskiego 10:05:00 -> Międzyrzecka 10:08:00  
920 | Międzyrzecka 10:12:00 -> Nowy Dom 10:16:00

Tabu dynamiczna wielkość listy Tabu

Total cost: 8306.878786135247 units

Execution time: 20.0202 seconds

Schedule:

```

120      | Nowy Dom 08:14:00 -> Na Niskich łąkach 08:21:00
3        | Na Niskich łąkach 08:22:00 -> Armii Krajowej 08:26:00
134      | Armii Krajowej 08:28:00 -> Armii Krajowej (Bogedaina) 08:29:00
143      | Armii Krajowej (Bogedaina) 08:30:00 -> GAJ 08:36:00
143      | GAJ 08:36:00 -> Działkowa 08:38:00
K        | Działkowa 08:39:00 -> Śliczna 08:41:00
143      | Śliczna 08:41:00 -> Uniwersytet Ekonomiczny 08:44:00
134      | Uniwersytet Ekonomiczny 08:44:00 -> Bzowa (Centrum Historii Zajeżdnia)
08:55:00
11        | Bzowa (Centrum Historii Zajeżdnia) 08:55:00 -> FAT 08:58:00
134      | FAT 08:58:00 -> Stalowa 09:03:00
4        | Stalowa 09:04:00 -> DWORZEC GŁÓWNY 09:14:00
N        | DWORZEC GŁÓWNY 09:14:00 -> GALERIA DOMINIKAŃSKA 09:18:00
13       | GALERIA DOMINIKAŃSKA 09:19:00 -> PL. GRUNWALDZKI 09:26:00
116      | PL. GRUNWALDZKI 09:28:00 -> Kochanowskiego 09:30:00
131      | Kochanowskiego 09:30:00 -> Śniadeckich 09:31:00
116      | Śniadeckich 09:31:00 -> Zacisze 09:32:00
131      | Zacisze 09:32:00 -> C.H. Korona 09:38:00
111      | C.H. Korona 09:38:00 -> Zacisze 09:43:00
131      | Zacisze 09:43:00 -> Śniadeckich 09:44:00
111      | Śniadeckich 09:44:00 -> Bujwida 09:48:00
131      | Bujwida 09:48:00 -> PL. GRUNWALDZKI 09:50:00
115      | PL. GRUNWALDZKI 09:50:00 -> Kliniki - Politechnika Wrocławska 09:52:00
10       | Kliniki - Politechnika Wrocławska 09:53:00 -> Chełmońskiego 09:58:00
143      | Chełmońskiego 10:05:00 -> Międzyrzecka 10:08:00
920      | Międzyrzecka 10:12:00 -> Nowy Dom 10:16:00

```

Tabu nieskończona wielkość listy Tabu

Total cost: 8306.878786135247 units

Execution time: 19.6260 seconds

Schedule:

```

120      | Nowy Dom 08:14:00 -> Na Niskich łąkach 08:21:00
3        | Na Niskich łąkach 08:22:00 -> Armii Krajowej 08:26:00
134      | Armii Krajowej 08:28:00 -> Armii Krajowej (Bogedaina) 08:29:00
143      | Armii Krajowej (Bogedaina) 08:30:00 -> GAJ 08:36:00
143      | GAJ 08:36:00 -> Działkowa 08:38:00
K        | Działkowa 08:39:00 -> Śliczna 08:41:00
143      | Śliczna 08:41:00 -> Uniwersytet Ekonomiczny 08:44:00
134      | Uniwersytet Ekonomiczny 08:44:00 -> Bzowa (Centrum Historii Zajeżdnia)
08:55:00
11        | Bzowa (Centrum Historii Zajeżdnia) 08:55:00 -> FAT 08:58:00
134      | FAT 08:58:00 -> Stalowa 09:03:00
4        | Stalowa 09:04:00 -> DWORZEC GŁÓWNY 09:14:00
N        | DWORZEC GŁÓWNY 09:14:00 -> GALERIA DOMINIKAŃSKA 09:18:00
13       | GALERIA DOMINIKAŃSKA 09:19:00 -> PL. GRUNWALDZKI 09:26:00
116      | PL. GRUNWALDZKI 09:28:00 -> Kochanowskiego 09:30:00
131      | Kochanowskiego 09:30:00 -> Śniadeckich 09:31:00
116      | Śniadeckich 09:31:00 -> Zacisze 09:32:00
131      | Zacisze 09:32:00 -> C.H. Korona 09:38:00
111      | C.H. Korona 09:38:00 -> Zacisze 09:43:00
131      | Zacisze 09:43:00 -> Śniadeckich 09:44:00
111      | Śniadeckich 09:44:00 -> Bujwida 09:48:00
131      | Bujwida 09:48:00 -> PL. GRUNWALDZKI 09:50:00
115      | PL. GRUNWALDZKI 09:50:00 -> Kliniki - Politechnika Wrocławska 09:52:00
10       | Kliniki - Politechnika Wrocławska 09:53:00 -> Chełmońskiego 09:58:00
143      | Chełmońskiego 10:05:00 -> Międzyrzecka 10:08:00
920      | Międzyrzecka 10:12:00 -> Nowy Dom 10:16:00

```